SHRINAV WORKSHOP

Practice Assignment

API Fundamentals & Postman Mastery

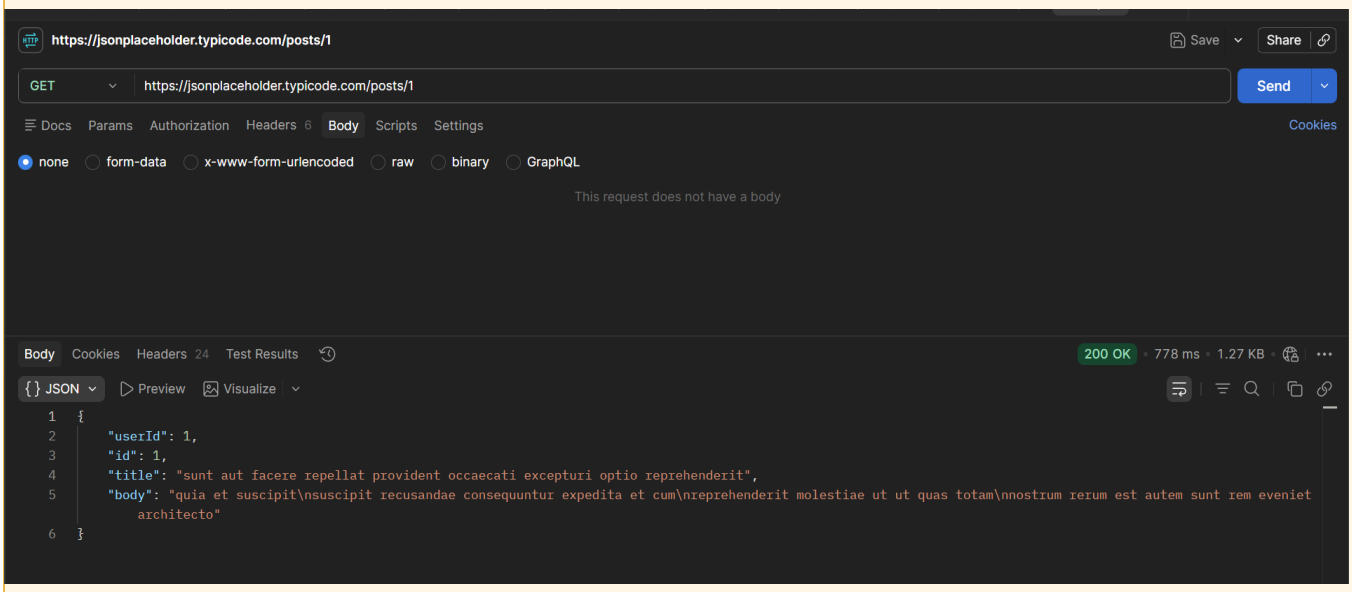
Complete every task in Postman using the JSONPlaceholder API. Read each instruction carefully — later tasks build on what you did in earlier ones.

Before you start: *you only need a browser and Postman. Base URL for every task: https://jsonplaceholder.typicode.com — no signup or API key required.*

Tasks

TASK 1 Reading Data Without Headers

a) Send a GET request to /posts/1 — leave the Headers and Body tabs completely empty.



b) Record the status code you get back.

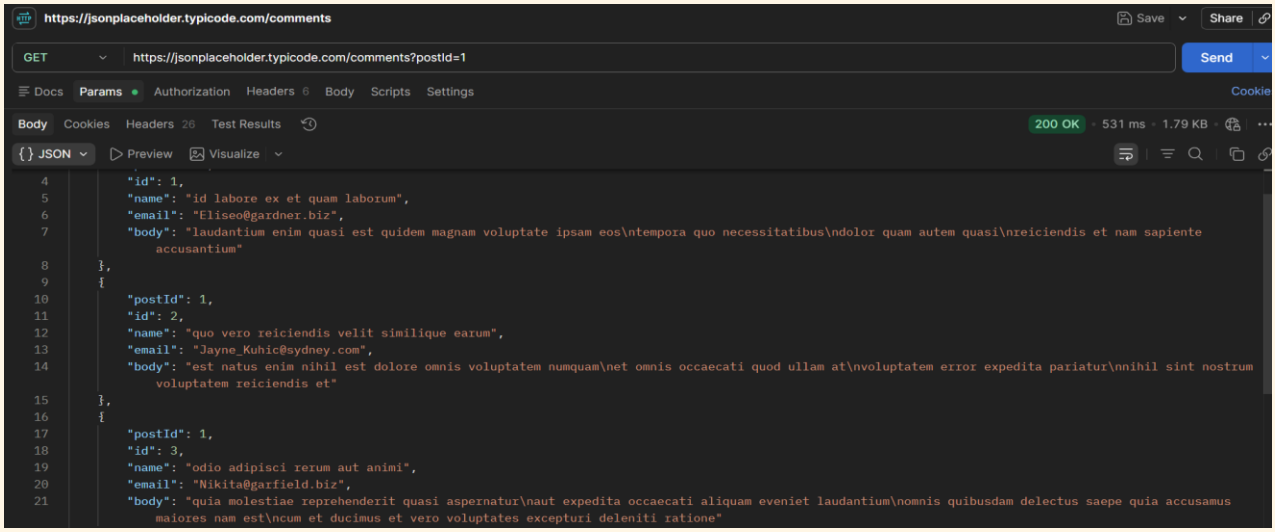
-> 200 OK

c) In one sentence, explain why this request works even with nothing in the Headers tab.

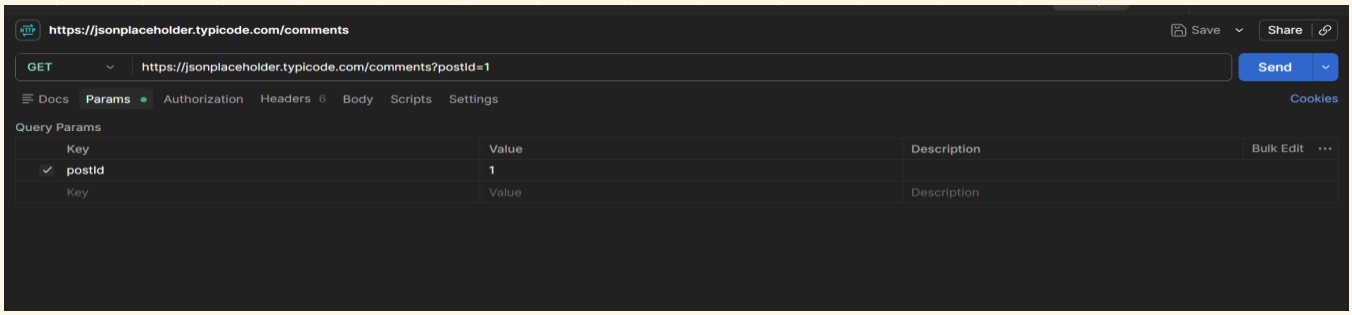
-> This request works because the server does not require any custom validation headers (like authentication keys) to access public data, and Postman automatically sends necessary default headers (like Host and User-Agent) behind the scenes.

TASK 2 Filtering With Query Parameters

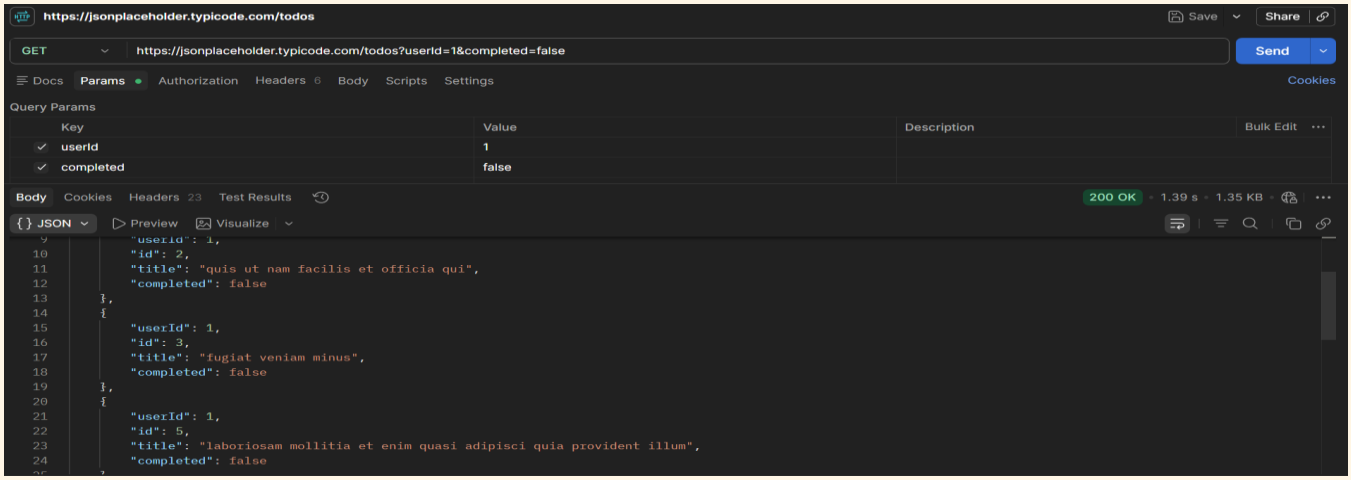
a) Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.



b) Confirm the URL automatically updates to include ?postId=1.



c) Now combine two filters at once on /todos: userId = 1 and completed = false.

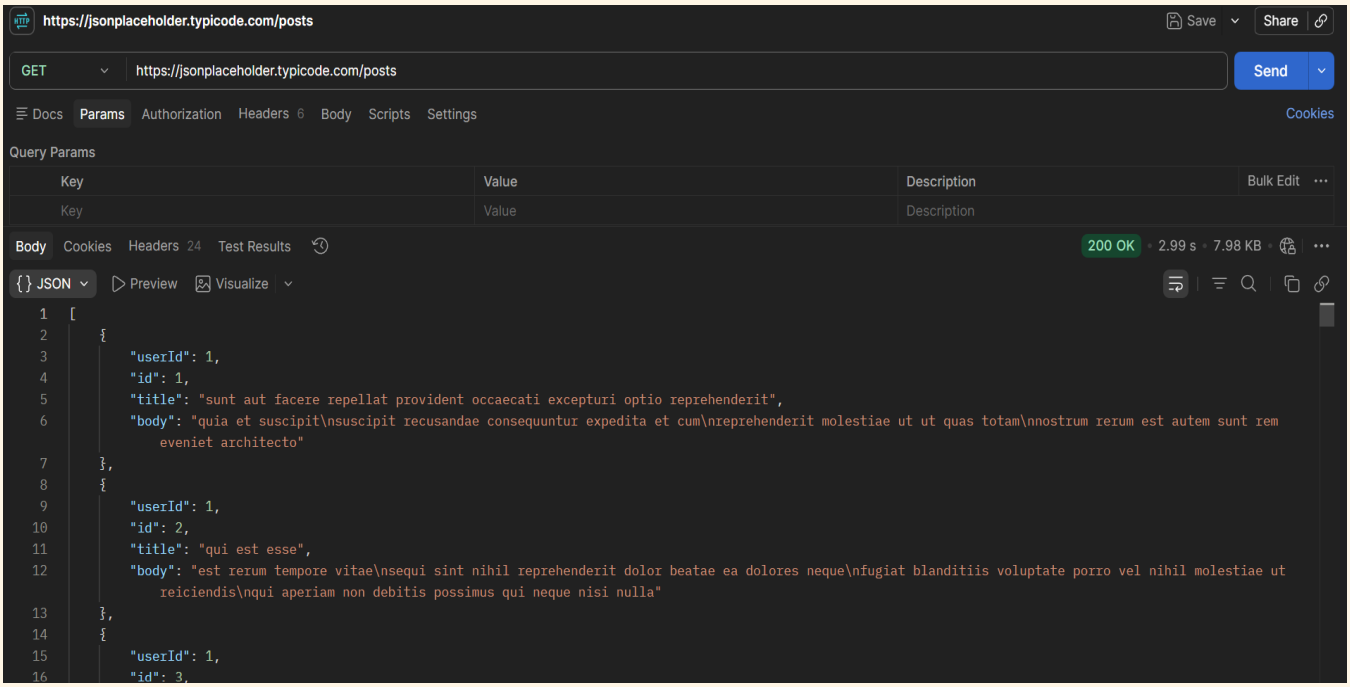


d) In one sentence, explain what changed in the results compared to not using any params.

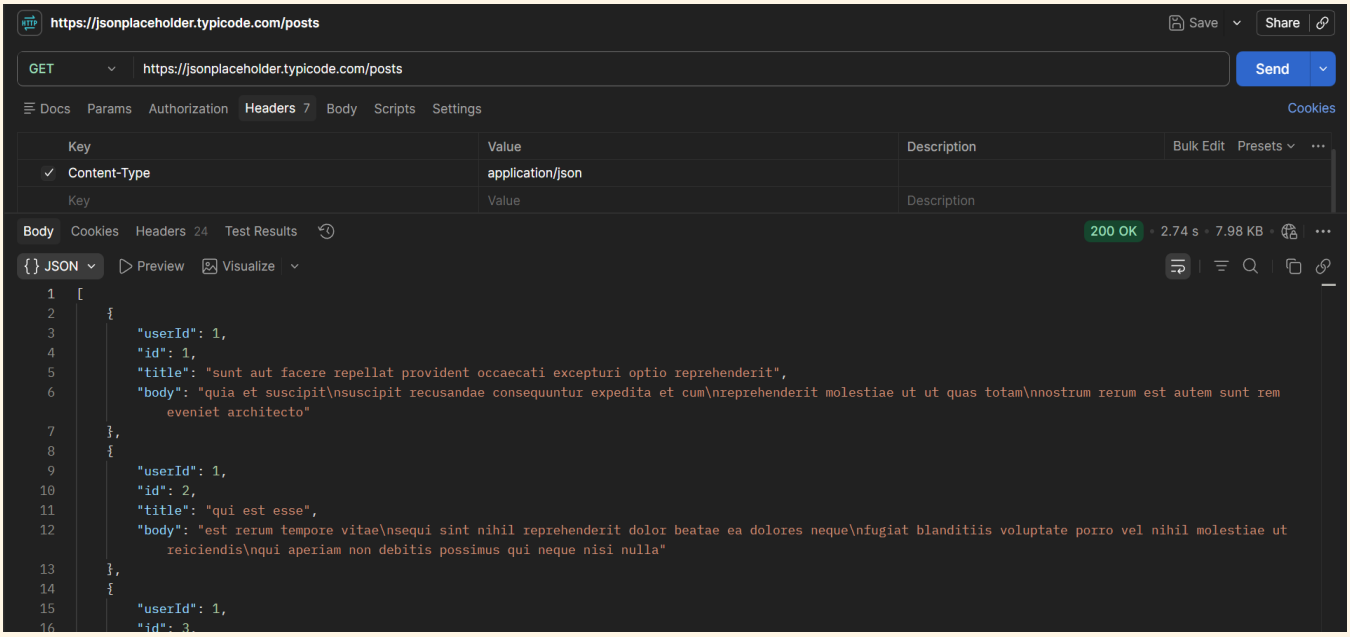
-> Using query parameters filters the dataset from the endpoint, returning only the specific records that match your criteria instead of delivering the entire un-ordered list of items.

TASK 3 Sending Data With Headers & a Body

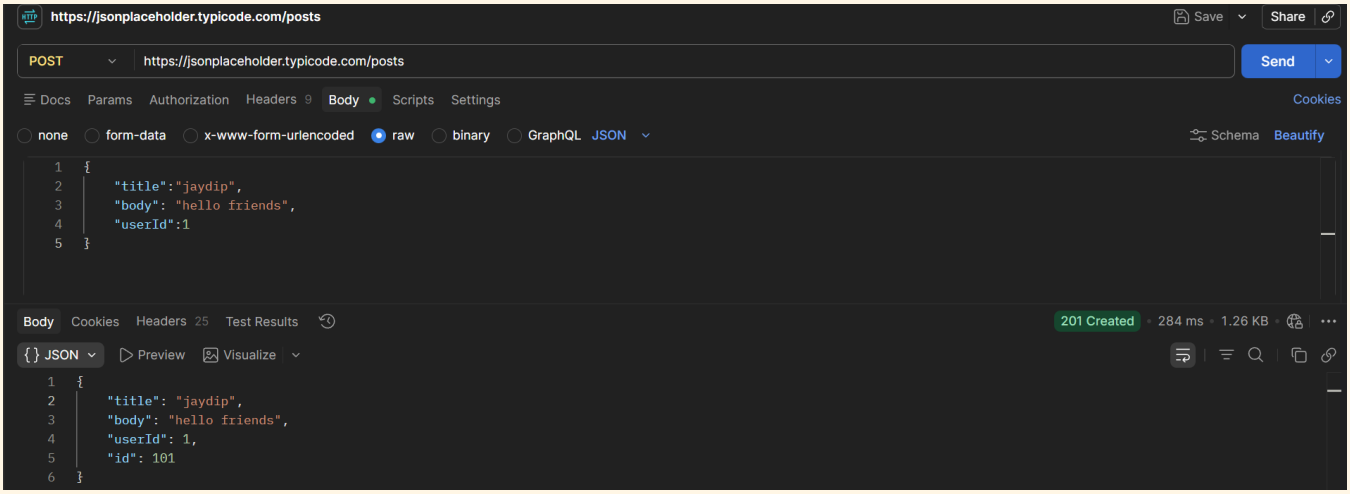
a) Send a POST request to /posts.



b) In the Headers tab, add: Content-Type: application/json



c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.



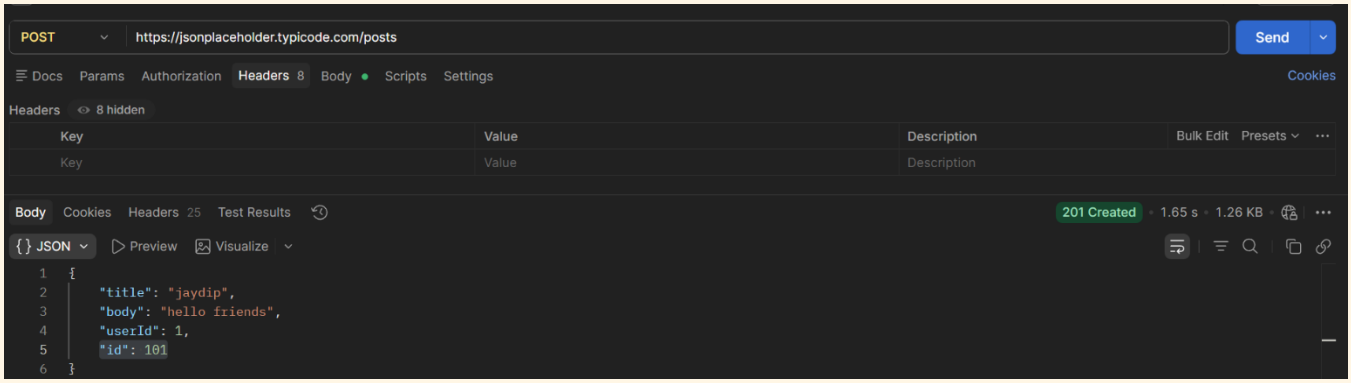
d) Record the status code and the new id the server assigned.

➔ 201Created

id: 101

TASK 4 Headers vs. No Headers

a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.



b) Compare the response to Task 3 — is it different? Record what you observe.

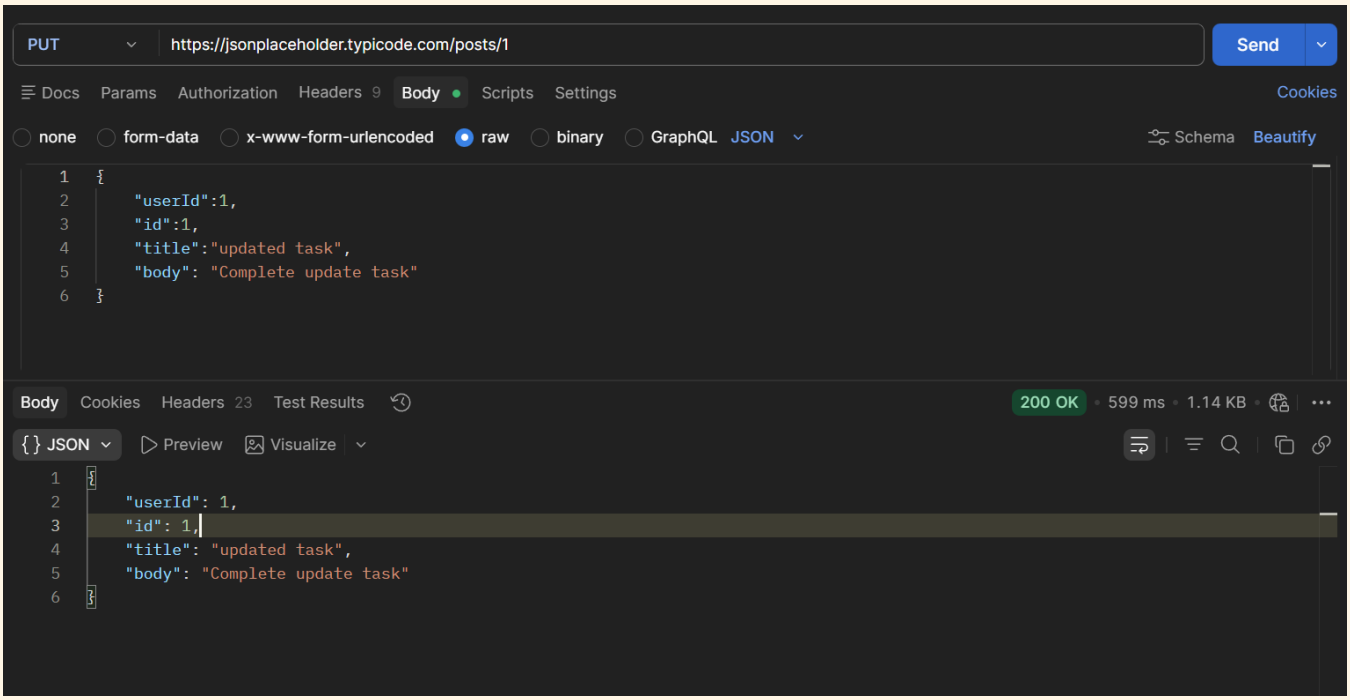
-> The response is almost same as Task 3. JSONplaceholder accepts the even without the content-Type header

c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

-> Headers tell the server what type of data is being sent in the request. The Content-Type header helps the server correctly understand the JSON body. JSONPlaceholder is a practice API, so it accepts the request without this header, but many real APIs would reject the request or return an error.

TASK 5 PUT vs. PATCH

a) Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).



b) Send a PATCH request to the same /posts/1 that changes only the title.

c) In one sentence, explain the difference between what PUT sent and what PATCH sent.

-> PUT request sends the entire resource object to completely replace it, whereas a PATCH request sends only the specific fields intended for a partial update.

Final Challenge — A Realistic Request Flow

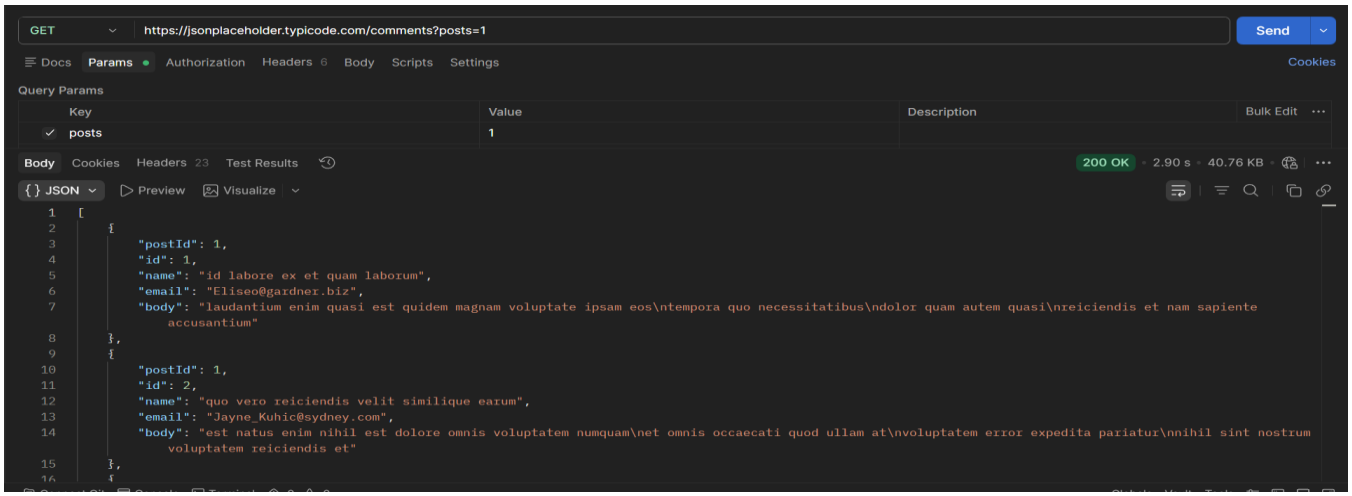
Concepts used: *this task combines query params, headers (with and without), and the request body — everything from Tasks 1 through 4 — in one flow.*

TASK 6 The Mini Feedback System

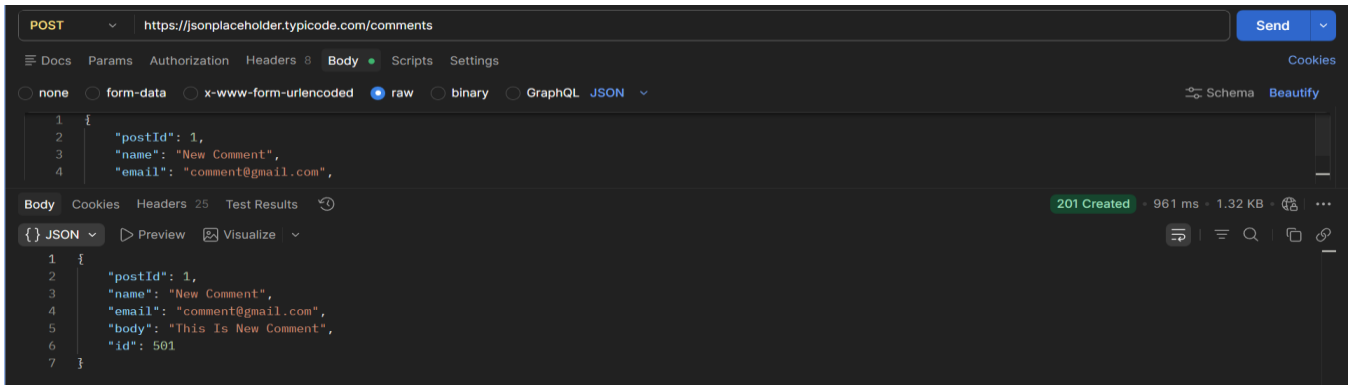
Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

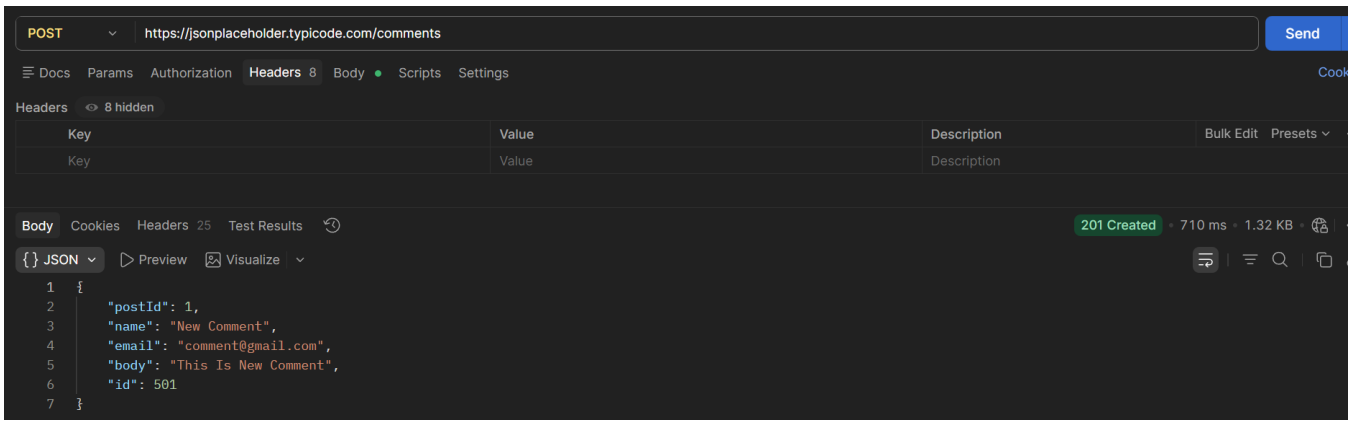
Complete the following steps, in order:

- a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.

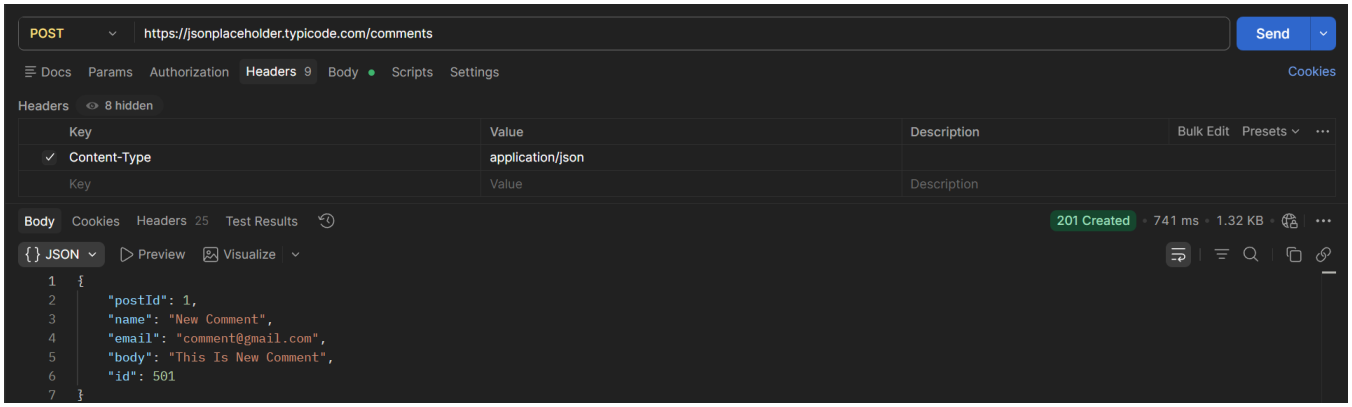


- b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.



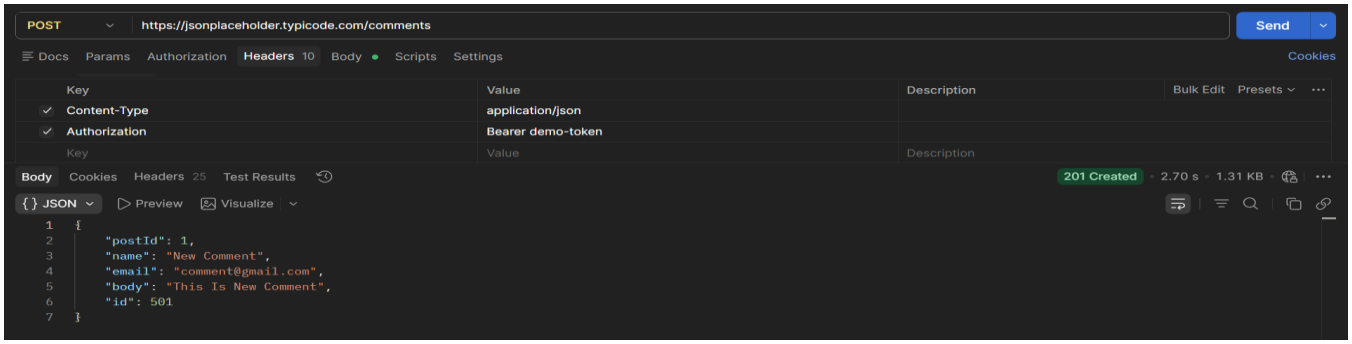


- c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.



Both request return the same response because JSONPlaceholder accepts requests even without the Content-Type header.

- d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.



- e) Record the status code for every step above (a–d) in a short table or list.

Step	Request	Status Code
a	GET /comments?postId=1	200 OK
b	POST /comments (No Header)	201 Created
c	POST /comments (Content-Type)	201 Created
d	POST /comments (Content-Type + Authorization)	201 Create

- f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

Ans: In a real API, the server would verify the Authorization token before accepting the request. If the token was missing or invalid, the server would return an error such as 401 Unauthorized or 403 Forbidden. Only authenticated users would be allowed to submit feedback.